```
  @Container                                                    ❷
  private static final KeycloakContainer keycloakContainer =
    new KeycloakContainer("quay.io/keycloak/keycloak:19.0")
      .withRealmImportFile("test-realm-config.json");

  @DynamicPropertySource                                        ❸
  static void dynamicProperties(DynamicPropertyRegistry registry) {
    registry.add("spring.security.oauth2.resourceserver.jwt.issuer-uri",
      () -> keycloakContainer.getAuthServerUrl() + "realms/PolarBookshop");
  }

  ...
}
```

❶ Activates automatic startup and cleanup of test containers

❷ Defines a Keycloak container for testing

❸ Overwrites the Keycloak Issuer URI configuration to point to the test Keycloak instance

The Keycloak test container is initialized via a configuration file I included in the code repository accompanying this book (Chapter12/12-end/catalog-service/src/test/resources/test-realm-config.json). Go ahead and copy it over to the src/test/resources folder of your Catalog Service project (catalog-service).

In production, we would call Catalog Service via Edge Service, which is responsible for authenticating users and relaying Access Tokens to the downstream applications. We now want to test Catalog Service in isolation and verify different authorization scenarios. Therefore, we need to generate some Access Tokens first, so that we can use them to call the Catalog Service endpoints under testing.

The Keycloak configuration I provided in the JSON file includes the definition of a test Client (`polar-test`) that we can use to authenticate users via a username and password directly, instead of going through the browser-based flow we implemented in Edge Service. In OAuth2, such a flow is called a *Password Grant*, and it’s not recommended for production use. In the following section, we’ll use it just for testing purposes.